

# Foundations of Algorithm SCS1308

Dr. Dinuni Fernando  
PhD

Senior Lecturer



# Cook's theorem

- SAT is NP-complete
- 3-SAT is NP-complete (1-SAT or 2-SAT is P)
- It is the first NP-complete problem
- Every NP problem reduces to SAT
- NP = P iff the SAT problem is a P problem

# NP Complete – Definition

- A problem B is called NP-complete if both of the following are true:
  1. B is in NP.
  2. For every other problem A in NP,

$$A \propto B$$

# Theory of NP completeness

- The theory of NP-completeness enables showing that these problems are at least as hard as *NP-complete* problems
- Practical implication of knowing a problem is NP-complete is that it is **probably** intractable (whether it is or not has not been proved yet)
- So any algorithm that solves it will probably be very slow for large inputs

# Satisfiability (SAT) problem

- Is there a truth assignment to the  $n$  variables of a logical expression in CNF which makes the value of the expression true?
- The answer is yes, if all clauses evaluate to true
- Otherwise, the answer is “no”

# SAT problem

- $p=T, q=F, r=T$  and  $s=T$  is a truth assignment for:

$$(p \vee q \vee s) \wedge (\neg q \vee r) \wedge (\neg p \vee r) \wedge (\neg r \vee s) \wedge (\neg p \vee \neg s \vee \neg q)$$

- Note that if  $q=F$  then  $\neg q=T$
- Each clause evaluates to true

# A verification algorithm for SAT

1. Check that the certificate **S** is a string of exactly  $n$  characters which are T or F.
  2. **while** (there are unchecked clauses) {  
    select next clause  
    **if** (clause evaluates to false) **return**( “no”) }
  3. **return** (“yes”)
- Is verification algorithm polynomial bound?
  - Satisfiability is in NP since there exists a polynomial bound verification algorithm for it

# Cook's theorem

- **SAT (at least 3-SAT) problem is NP complete**
  - Cook proved that SAT is NP and every problem in NP reduces to SAT
  - First problem proved to be NP complete
  - Proof idea: encode the workings of a Nondeterministic Turing machine for an instance  $I$  of problem  $X \in NP$  as a SAT formula so that the formula is satisfiable iff the nondeterministic Turing machine accepts the instance  $I$

- After Cook's theorem, many NP-complete problems are found
  - E.g., 3-SAT  $\propto$  Clique, 3-SAT  $\propto$  Hamiltonian Cycle Decision Problem, 3-SAT  $\propto$  3-coloring, ...
- More NP-complete problems are found from NP complete problems that are not 3-SAT
  - E.g., Hamiltonian circuit  $\propto$  Traveling Salesperson, Clique  $\propto$  vertex cover ...

# Shortcut for NP-completeness Proofs

- To prove a problem  $L$  is NP-complete:
  - Prove  $L \in \text{NP}$ .
  - Choose  $L' \in \text{NPC}$ , and show  $L' \propto L$ 
    - Transitivity

# Reductions

For example, let's discuss how to:

- Reduce 3-SAT to Clique
- Reduce Clique to Vertex Cover
- Reduce 3-SAT to Hamiltonian Cycle

# Clique

- Show clique is a NP-complete problem via reduction

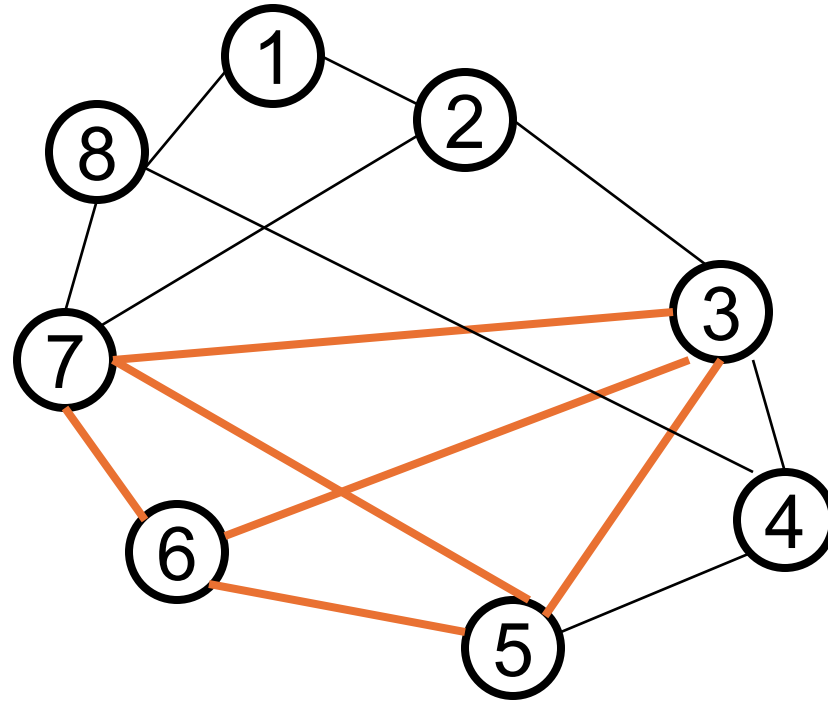
# The Clique Problem

- A *clique* is a complete undirected graph where every vertex is connected to every other vertex.

## CLIQUE

- Input: An undirected graph  $G$  and a positive integer  $k$ .
- Output: YES iff a clique of size  $k$  exists in  $G$ .

# Clique example



- $G$  contains a clique of 4 (with vertices 3, 5, 6, 7)

# The Clique Problem

- **Theorem:** Show CLIQUE is NP-complete.
- **Proof:**
- **Step 1.** CLIQUE  $\in$  NP

Given a certificate that contains a set of  $k$  vertices  $V' \subseteq V$ , we can check if  $V'$  forms a clique by checking for every pair of nodes  $u, v \in V'$  that  $(u,v) \in E$

- Clearly, this can be done in polynomial time.

# The Reduction

## Step 2. Selection

3-CNF-SAT which is NP-Complete.

## Step 3. Mapping

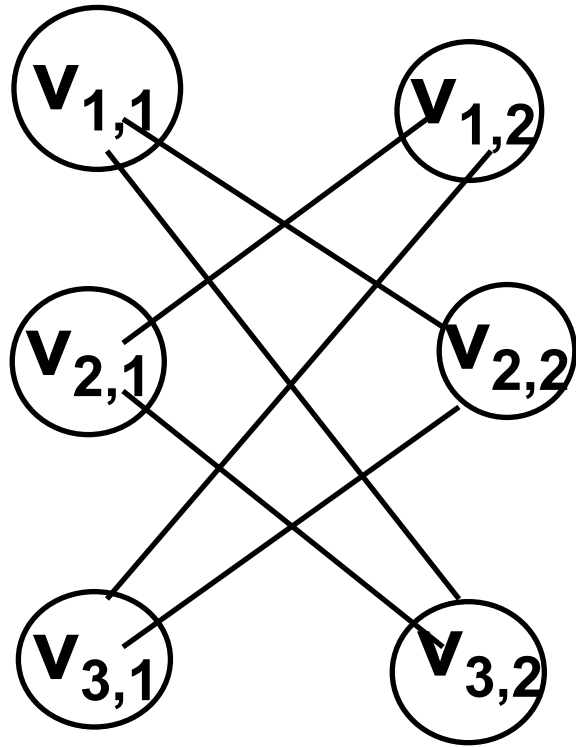
For a formula  $C_1 \wedge \dots \wedge C_k$  such that  $C_r = l_{1,r} \vee l_{2,r} \vee l_{3,r}$  we construct a graph  $G$  with vertices  $v_{1,r} v_{2,r} v_{3,r}$  for  $r = 1, \dots, k$ , where  $v_{i,r}$  represents the literal  $l_{i,r}$

# The Reduction

We put an **edge** between  $v_{i,r}$  and  $v_{j,s}$  if both of the following hold:

1.  $r \neq s$  and
2.  $l_{i,r}$  is not the negation of  $l_{j,s}$ .

$$(x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$$



**k=2**

**The graph has 6  
cliques of size 2**

# Step 4a. Yes for 3-SAT implies yes for clique

- Assume formula satisfiable.
- With the satisfying assignment each clause contains at least 1 literal that is assigned 1.
- Since each literal from each clause is a vertex in the graph, if we pick out a literal that is assigned 1 from each of the  $k$  clauses, we get  $k$  vertices in the graph.

## Step 4a. Yes for 3-SAT implies yes for Clique

- This set of  $k$  vertices is a clique.
  - For any two vertices, the corresponding literals are from different clauses, and are both assigned 1, so they cannot be complements of a single variable
  - Thus, there is an edge between any two such vertices.

## Step 4b. Yes for Clique implies yes for 3-SAT

- Assume  $G$  has a clique  $V'$  of size  $k$
- No edge connects vertices in the same clause, so each of  $k$  triples has exactly one vertex in  $V'$
- Assign 1 to each literal in  $V'$  without getting an inconsistent assignment (why?), and assign arbitrary values to the rest of the variables
- For this assignment, each clause is satisfied and thus the answer for 3-SAT is yes

## Step 5. Reduction is polynomial

- Step 5. The reduction is polynomial.
  - The formula is read and  $3k$  vertices are generated in  $O(k)$  steps. Then, each pair of literals from two different clauses is checked and an edge is added if the literals are not complimentary.
  - The reduction is  $O(k^2)$

Show: Clique is NPC by Reducing 3SAT to Clique

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$$

# Reduction of 3SAT to Clique

- This shows clique is NP hard since 3SAT is NP complete.
- **3SAT problem** – Boolean formula in CNF where each clause contains exactly 3 literals.

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$$

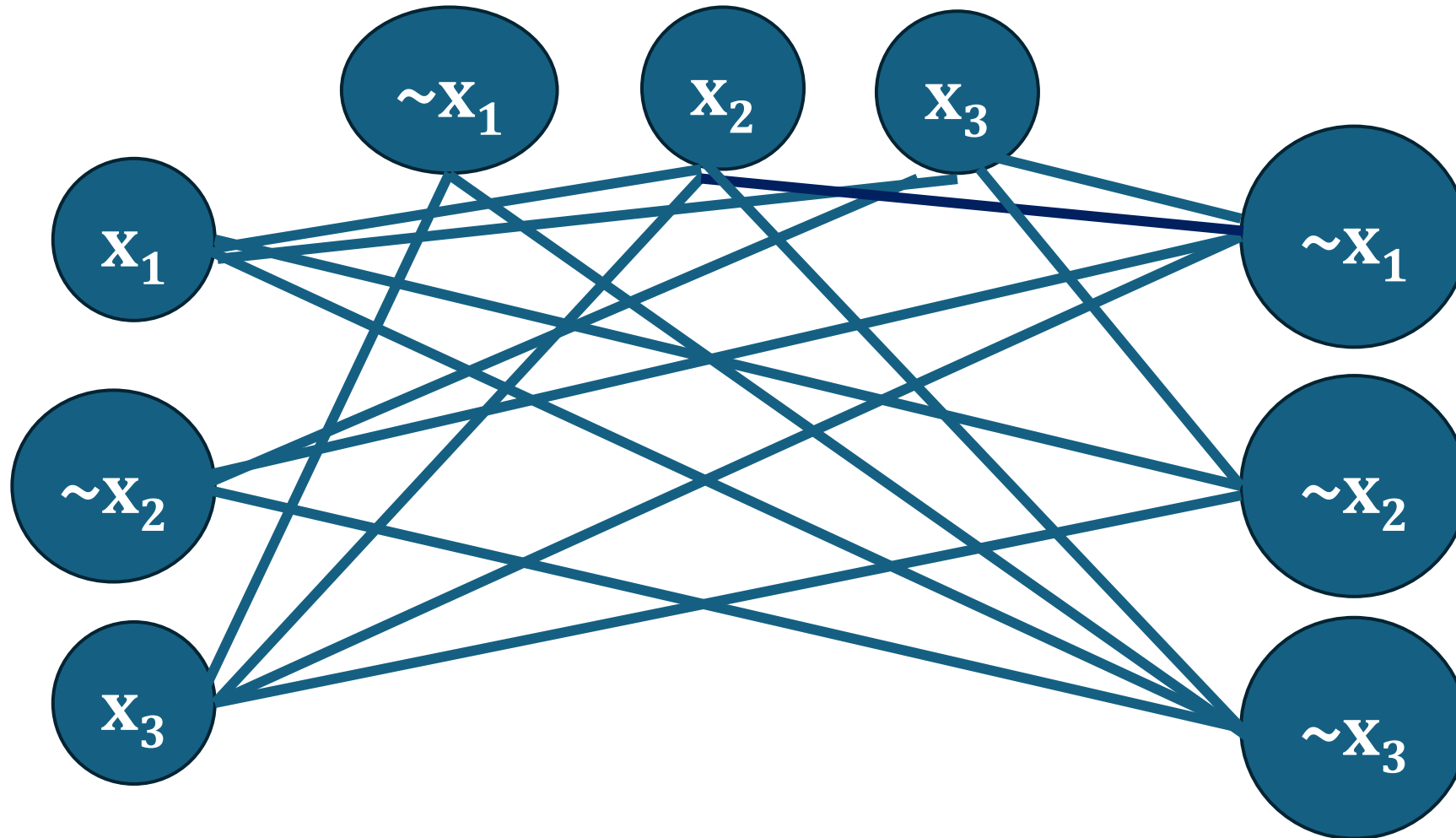
- Goal is to determine if there exists an assignment of variables  $x_1, x_2, x_3$  such that  $F$  evaluates to be true.
- **Clique problem** –  $G = (V, E)$  and integer  $k$ , determine whether  $G$  contains a clique of size  $k$ . A clique is a subset of vertices  $v' \subseteq v$  such that every pair of vertices in  $v'$  is connected by an edge.

# Reduction of 3SAT to Clique

- 3 SAT formula  $F$  can have  $m$  clauses and each clause  $c_i$  contains exactly 3 literals.
- Construct graph  $G$  = for each literal  $l$  in each clause  $c_i$  create a vertex, 3 vertices for each clause and  $3m$  vertices in total.
- Add edge between two vertices iff:
  - If they are from different clause
  - If they are not contradictory
- Set  $k = m$  such that no. of clauses in 3SAT. Goal is to find a clique of size  $m$  in  $G$ .

# Reduction of 3SAT to Clique

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$$

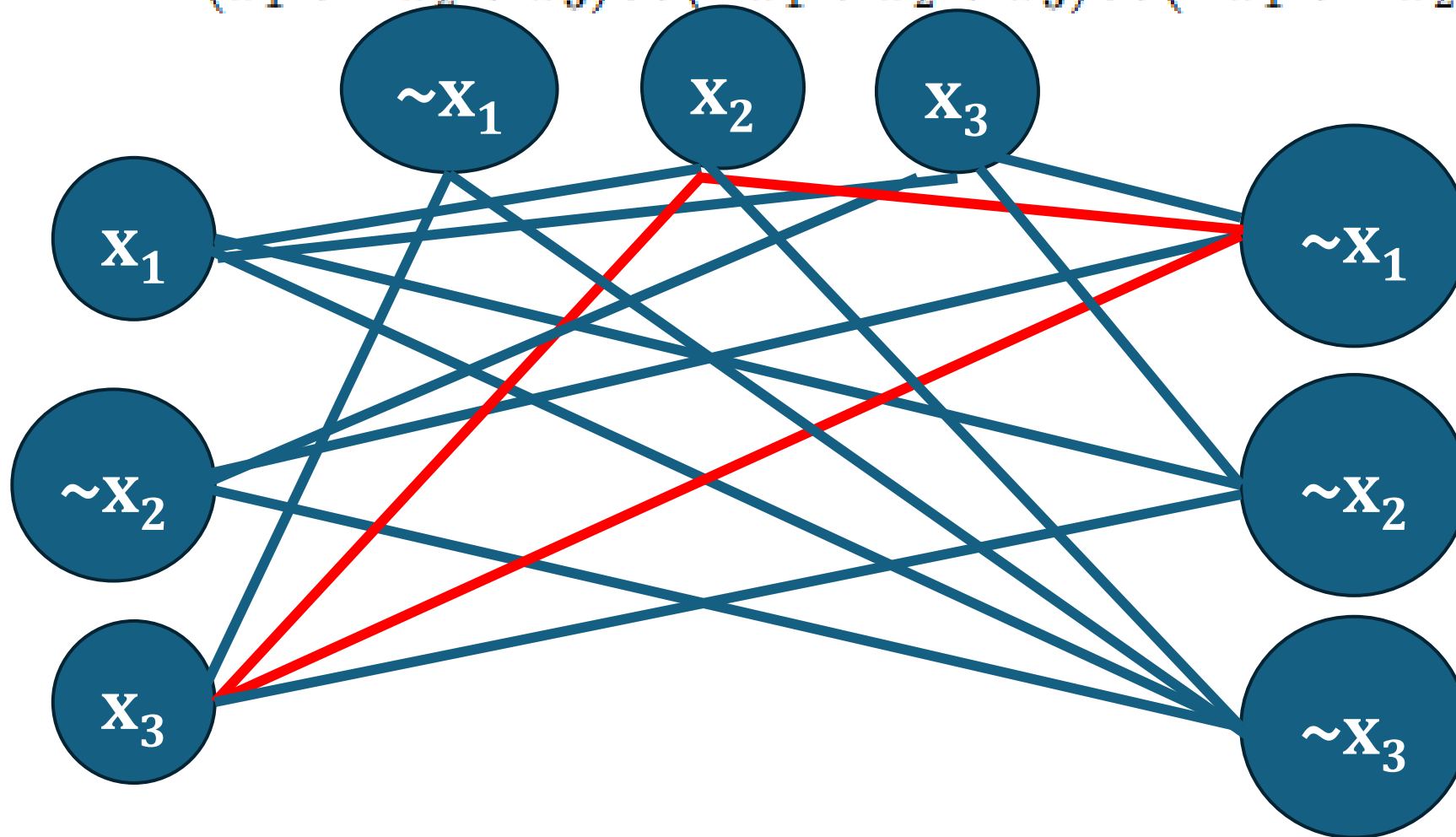


# Reduction of 3SAT to Clique

- Construction of the graph can be performed in polynomial time.
- If  $F$  is satisfiable, let  $A$  be a satisfying assignment, where we select from each clause a literal that is true in  $A$ , such that we construct a set  $S$  where  $|S| = k$  since there are no two literals from same clause.
- All of them simultaneously true for all corresponding nodes and they are connected to each other forming  $k$  clique. In our case its 3 clique. Hence graph has  $k$  clique.

# Reduction of 3SAT to Clique

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$$



# Reduction of 3SAT to Clique

- There exist a clique such that corresponding assignment of variables satisfies all clauses in F.
- Conversely If F is satisfiable, there will be a clique of size 3 in G
- $X_1 = \text{False}$
- $X_2 = \text{True}$
- $X_3 = \text{True}$

$$\underset{\text{T}}{F} = (\underset{\text{T}}{x_1} \vee \neg \underset{\text{T}}{x_2} \vee \underset{\text{T}}{x_3}) \wedge (\neg \underset{\text{T}}{x_1} \vee \underset{\text{T}}{x_2} \vee \underset{\text{T}}{x_3}) \wedge (\neg \underset{\text{T}}{x_1} \vee \neg \underset{\text{T}}{x_2} \vee \neg \underset{\text{T}}{x_3})$$

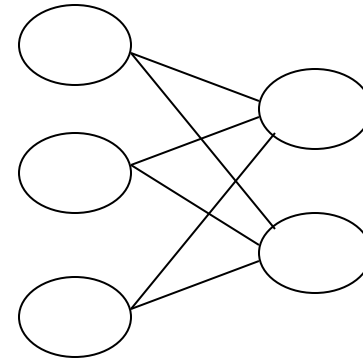
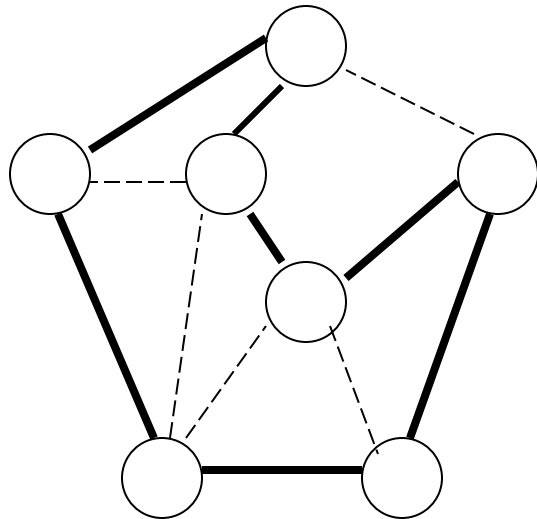
So F is True.

# Decision Problems

- A *decision* problem answers *yes or no* for a given input
- Examples:
  - Given a graph  $G$ , is there a path from  $s$  to  $t$  of length at most  $k$ ?
  - Does graph  $G$  contain a Hamiltonian cycle?
  - Given a graph  $G$ , is it bipartite?
  - For a 0-1 knapsack problem, is there a solution whose benefit is \$100 or more?

# A decision problem: HAMILTONIAN-CYCLE

- A *Hamiltonian cycle* of a graph  $G$  is a cycle that visits each vertex of the graph (except for the starting node) exactly once.
- Problem: Given a graph  $G$ , does  $G$  has a Hamiltonian cycle? Show Hamiltonian cycle is NPC



# Hamiltonian Cycle is NP-complete

- To prove that Hamiltonian Cycle is NP-complete, we must show:
  - A.  $\text{HamCycle} \in \text{NP}$  (a given cycle can be verified in polynomial time), and
  - B.  $3\text{-SAT} \leq_p \text{HamCycle}$  (polynomial-time reduction from 3-SAT to HamCycle).

# 1. Hamiltonian cycle is in NP

- Certificate : ordered list of vertices
- Verifier
  - First check if there are duplicated vertices
  - Then check all consecutive pairs are joined by some edges belong to the graph.
  - Polynomial time

# Reduce 3SAT to Hamiltonian cycle

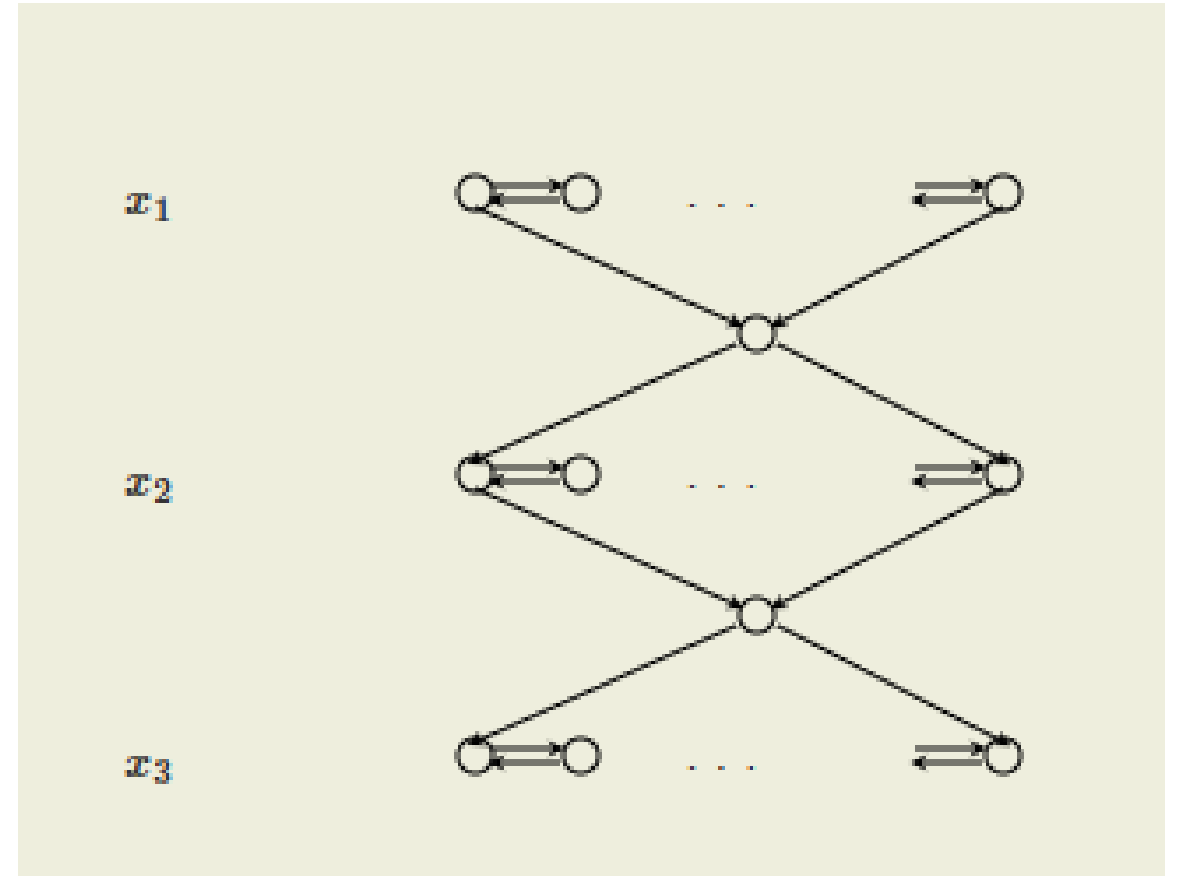
- 3SAT
  - Input variables  $x_1, x_2, x_3, \dots, x_n$
  - Clauses  $c_1, c_2, \dots, c_m$
  - Example  $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$
  - Yes instance if some assignment of variables can satisfy the formula.
- HamCycle
  - Input : directed graph  $G = (V, E)$
  - Yes, if contains cycle such that visits each vertex exactly once.

# Reduction

- Each literal corresponds to a variable
- Each time we can go left or right
  - Left to right means  $x_1$  is true
  - Right to right means  $x_1$  is false
- $2^n$  different possible truth assignments correspond to  $2^n$  distinct possible cycles of the form
  - $S \rightarrow \text{down left/right} \rightarrow \text{right/left} \rightarrow \dots \rightarrow t \rightarrow s$
- Having a Hamiltonian cycle does NOT yet guarantee that all clauses are satisfied.

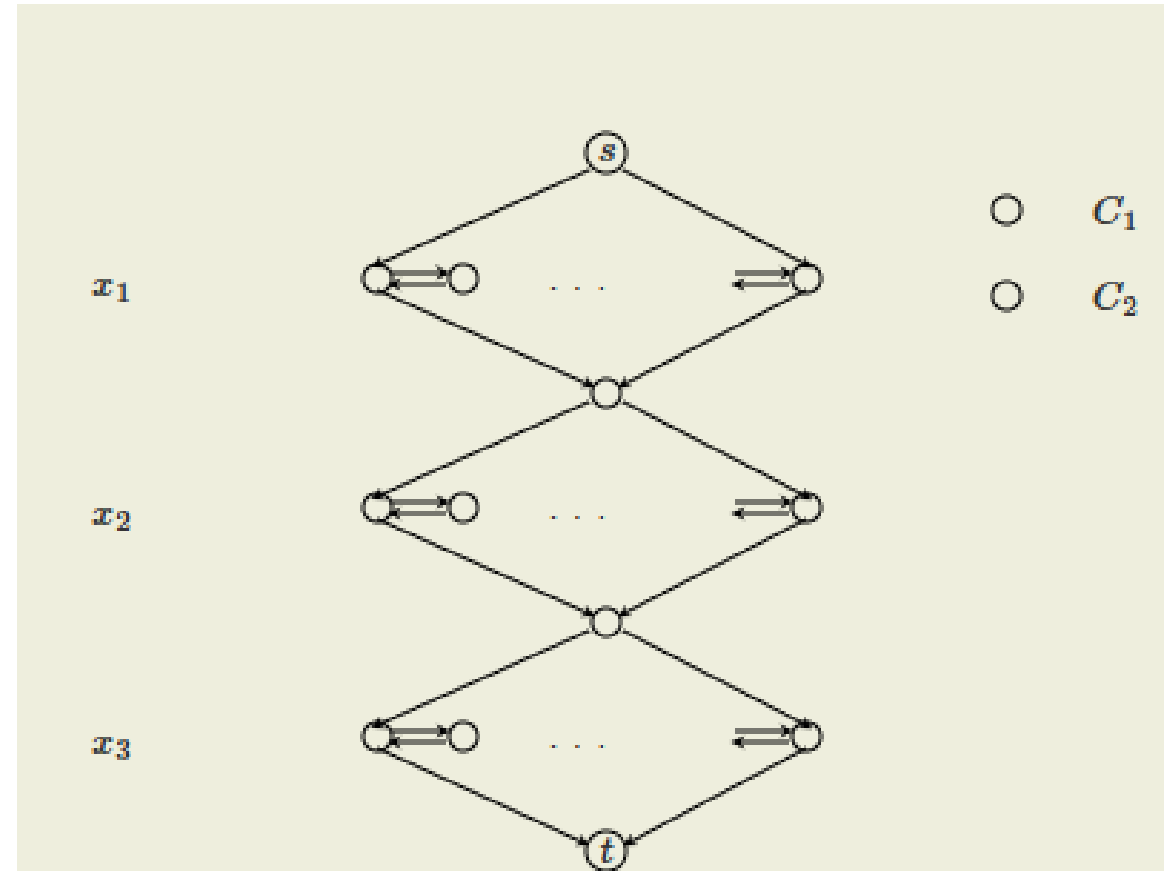
# Graph construction

- Each variable from 3SAT represent by its own row of nodes (gadget).
- Row will be connected to left to right. Representing CNF expression with three variables  $x_1, x_2, x_3$
- Very top node  $S$  (start) is added, bottom node  $t$  is added.



# Graph construction

- Representing clauses using nodes.
- If there are 2 clauses  $c_1, c_2$  we create 2 nodes for each clause.
- How many nodes need to be added for variable gadget in each row !



# Graph construction : variable gadget

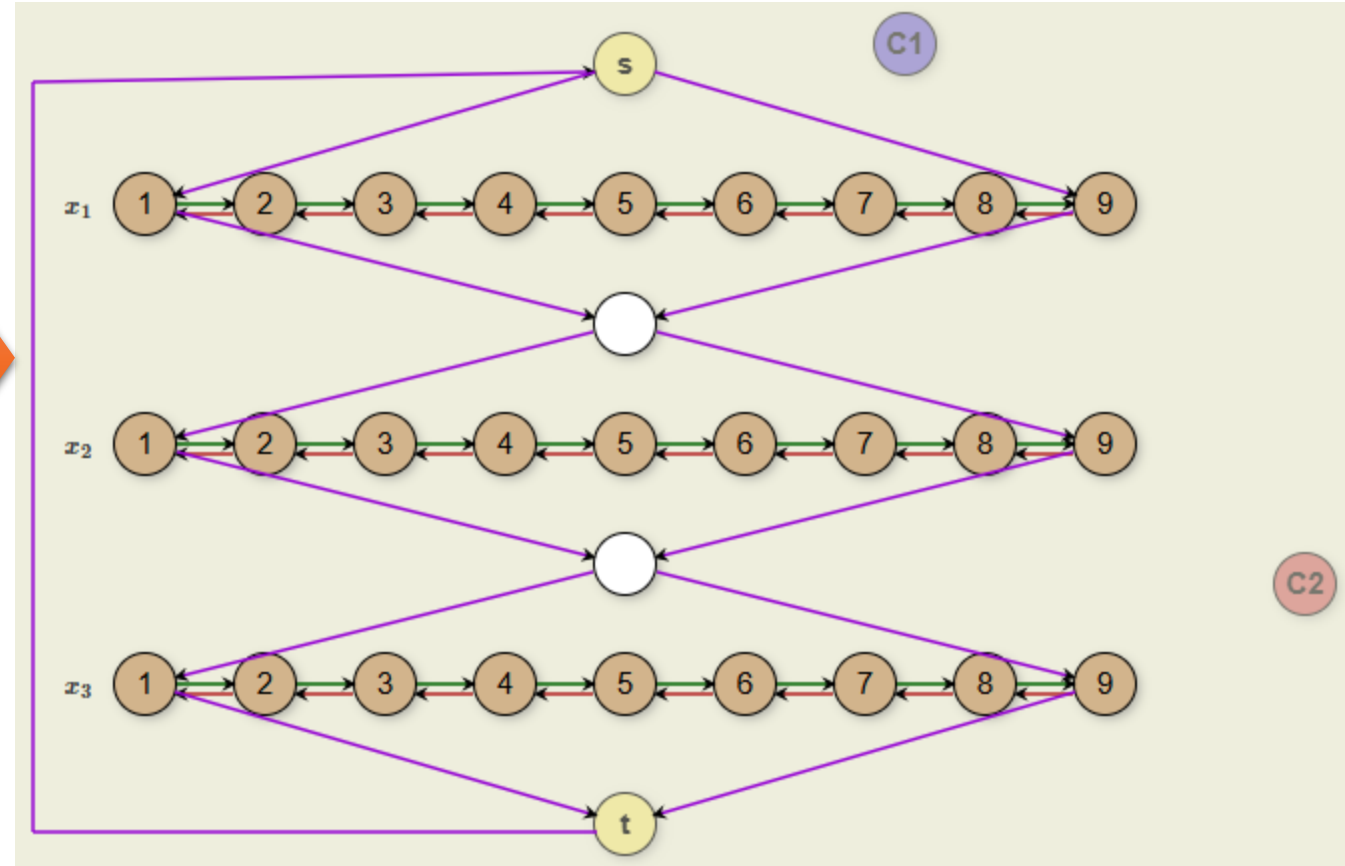
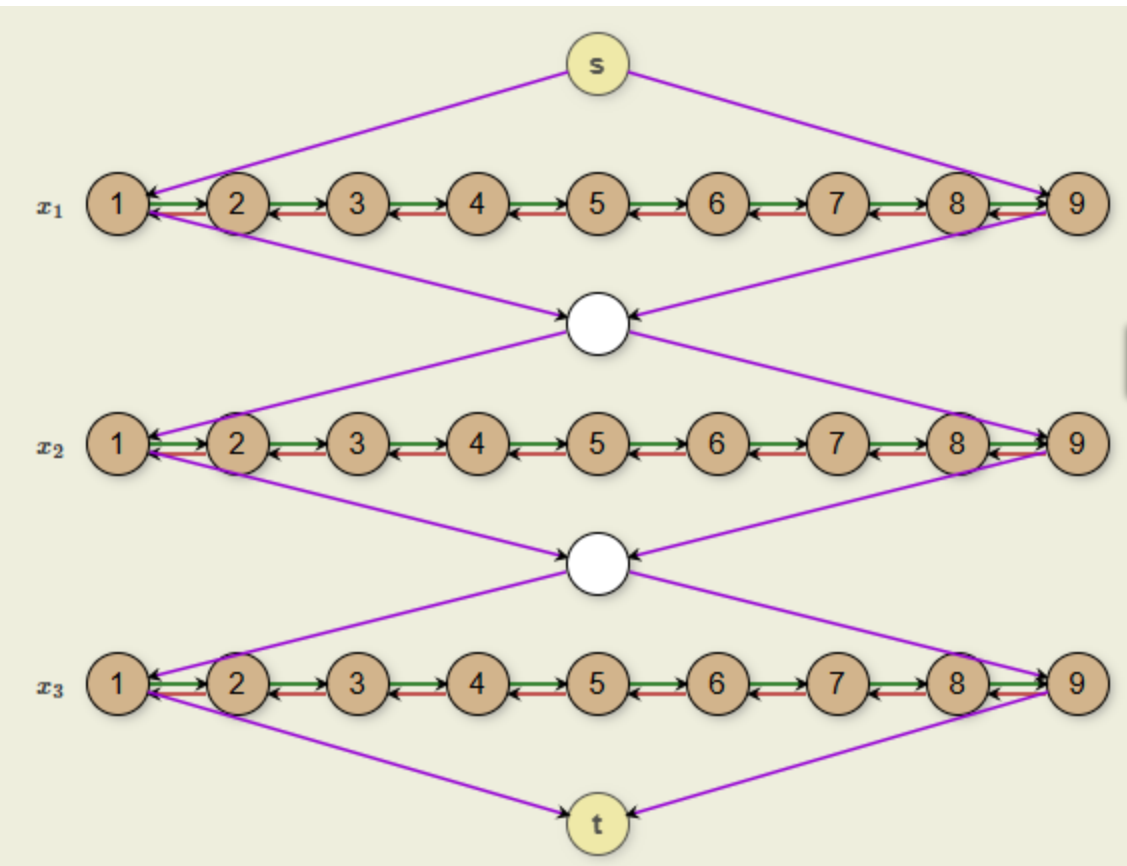
- Each variable is represented as a row of nodes (a variable gadget).
- Since there are  $k$  clauses, and each clause has 3 literals, the variable gadget must support connections related to these clauses.
- This leads to a construction where each variable row contains  $3k$  internal nodes plus a small number of additional nodes (entry/exit nodes), giving  $3k+3$  nodes per variable row.

# Graph construction : variable gadget

- Why 3 connections per row ?
- For each variable row
  - 1 connection for the positive literal
  - 1 connection for the negated literal
  - 1 auxiliary node or padding for traversing logical paths in gadget
- Additional 3 nodes (Entry, Exit and Control) ensure that
  - Enters the gadget correctly from other parts of the graph.
  - Traverses the True or False path for the variable
  - Exists the gadget correctly.

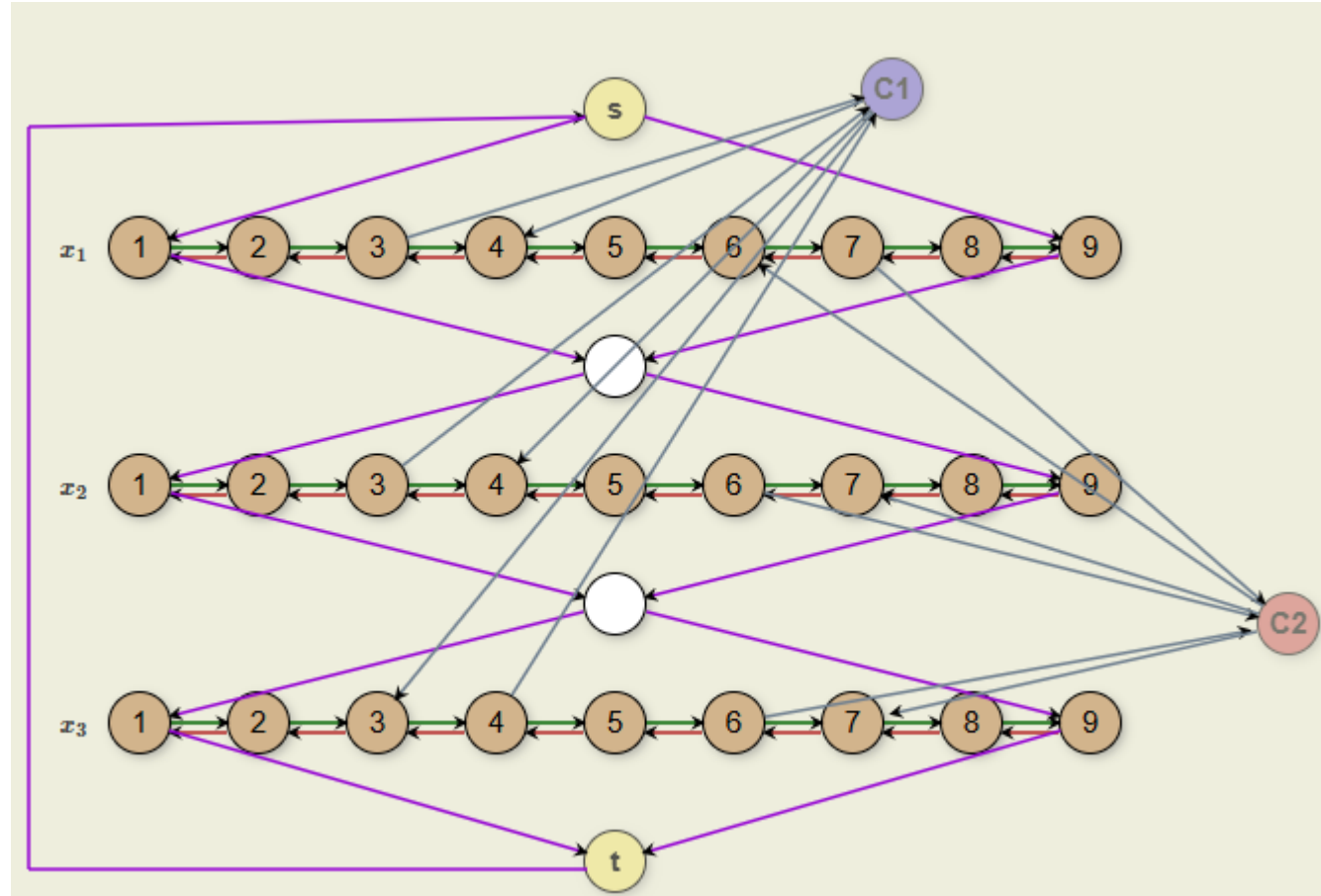
# Graph construction

$$(x_1 + x_2 + \overline{x_3}) \cdot (\overline{x_1} + x_2 + x_3).$$



# Completed graph construction

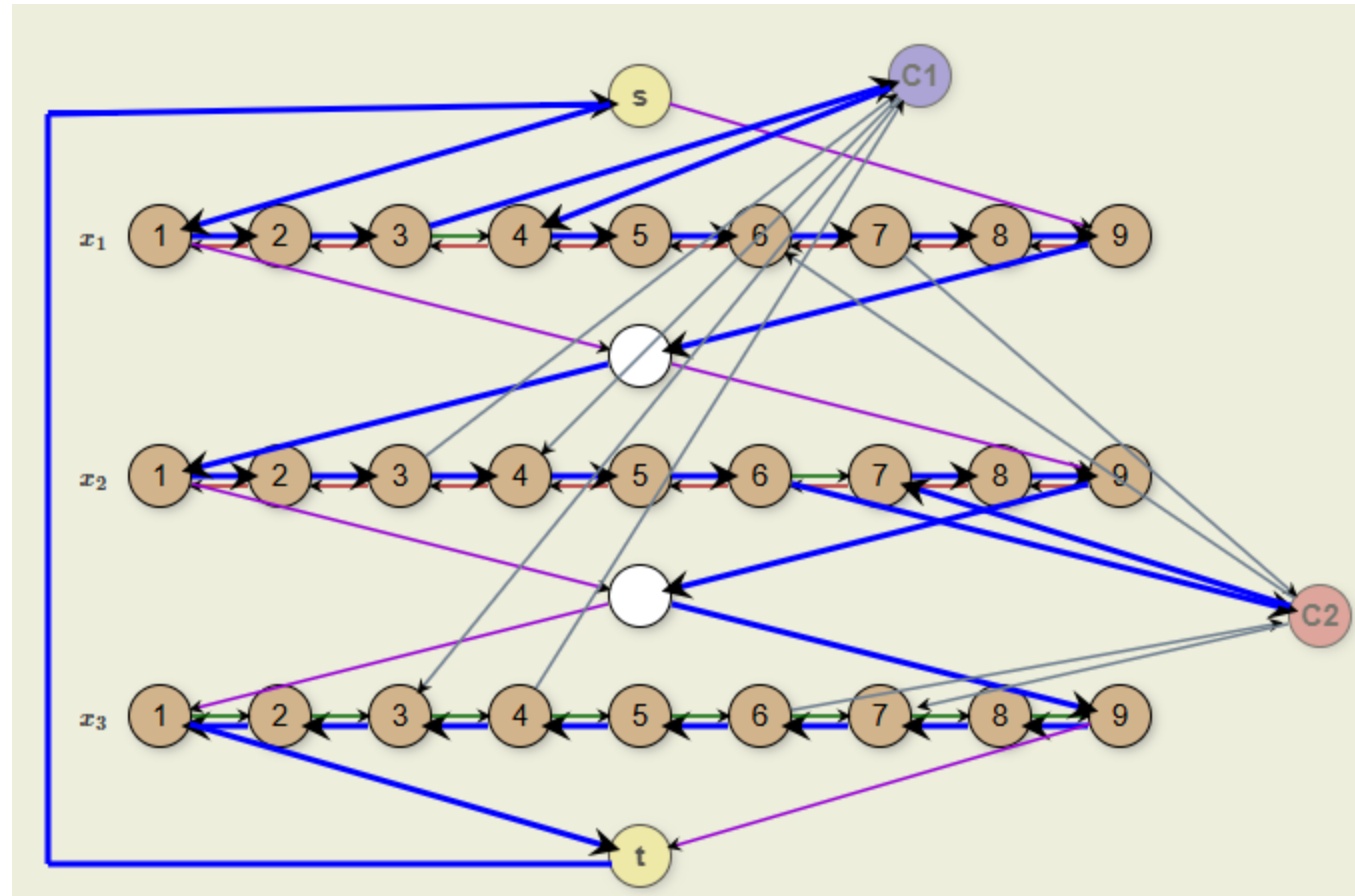
$$(x_1 + x_2 + \overline{x_3}) \cdot (\overline{x_1} + x_2 + x_3).$$



# 3SAT and Hamiltonian cycle

1. If there exist a Hamiltonian cycle  $H$  in the graph  $G$ 
  - If  $H$  traverses  $P_1$  from left to right, assign  $x_i = \text{True}$
  - If  $H$  traverses  $P_1$  from right to left, assign  $x_i = \text{False}$
  - The assignment obtained here satisfies the given 3CNF.
2. If there exists a satisfying assignment for the 3CNF.
  - Select the path that traverses  $P_i$  from left to right if  $x_i = \text{true}$  or right to left if  $x_i = \text{false}$ .
  - Connect source to  $p_1$ ,  $p_n$  to target and  $p_i$  to  $p_{i+1}$  appropriately to maintain the continuity of the path
  - Connect the target to source to complete the cycle.

# Hamiltonian cycle in the constructed graph



$$(x_1 + x_2 + \overline{x_3}) \cdot (\overline{x_1} + x_2 + x_3).$$

# How to prove problem is NP-Complete

1. The problem is in NP

This means any solution to the problem can be verified in polynomial time

2. The problem is NP-hard

This means every problem in NP can be reduced to this problem in polynomial time.

# Steps to Prove NP-Completeness

## 1. Show the Problem is in NP

Identify the certificate (solution) for the problem.

Demonstrate that this certificate can be verified in polynomial time.

## 2. Show the Problem is NP Complete

Choose a known NP-complete problem (e.g., **3-SAT**, **Hamiltonian Cycle**, **Vertex Cover**, etc.).

Provide a **polynomial-time reduction** from the known NP-complete problem to the target problem.

# Proving Hamiltonian Cycle is NP-Complete

- **Step 1: Hamiltonian Cycle is in NP**
- **Certificate:** A sequence of vertices representing a Hamiltonian cycle (visiting each vertex exactly once and returning to the starting vertex).
- **Verification:**
  - Check if the sequence contains all vertices exactly once  $O(V)$
  - Check if there is an edge between consecutive vertices in the sequence  $O(V)$
  - Total verification time is  $O(V^2)$ , which is polynomial.
- **Conclusion:** Hamiltonian Cycle is in NP.

- **Step 2: Hamiltonian Cycle is NP-Complete**
- Reduce a known NP-complete problem (e.g., **3-SAT**) to Hamiltonian Cycle in polynomial time.
- Sketch of the reduction:
  - Construct a graph  $G$  from the 3-SAT formula such that:
    - Each clause is represented by a subgraph.
    - Each variable is represented by a subgraph.
    - Connections between variable and clause subgraphs enforce the logical constraints.
  - A Hamiltonian cycle in  $G$  exists if and only if the 3-SAT formula is satisfiable.
- Prove that the reduction process takes polynomial time.
- **Step 3: Conclude NP-Completeness**
- Since Hamiltonian Cycle is in NP and NP-hard, it is NP-complete.

# Converting to decision problems

- Optimization problems can be converted to decision problems (typically) by adding a bound  $B$  on the value to optimize, and asking the question:
  - Is there a solution whose value is at most  $B$ ? (for a minimization problem)
  - Is there a solution whose value is at least  $B$ ? (for a maximization problem)

# An optimization problem: traveling salesman

- Given:
  - A finite set  $C = \{c_1, \dots, c_m\}$  of cities and
  - A distance function  $d(c_i, c_j)$  of non-negative numbers
- Find the length of the **minimum** distance tour which visits every city exactly once and comes back to the starting city

# A decision problem for traveling salesman

- Given a finite set  $C = \{c_1, \dots, c_m\}$  of cities, a distance function  $d(c_i, c_j)$  of non-negative numbers and a bound  $B$
- Is there a tour of all the cities (in which each city is visited exactly once) with total length **at most  $B$** ?
- There is no known polynomial bound algorithm for TS.

# Relation between an optimization problem and the decision problem

- If we have a solution to the optimization problem, we can compare the solution to the bound and answer “yes” or “no”
- Therefore, if the optimization problem is tractable so is the decision problem
- If the decision problem is “hard” the optimization problem is also “hard”
  - If the optimization is easy, then the decision problem is easy

# The class P

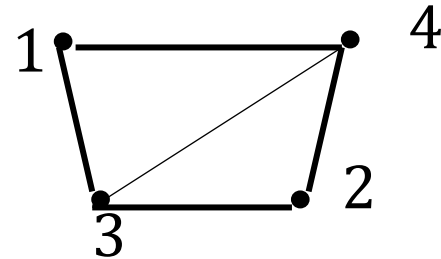
- **P is the class of decision problems that are polynomial bound**
- Is the following problem in P?
  - Given a weighted graph  $G$ , is there a spanning tree of weight at most  $B$ ?
- The decision versions of problems such as shortest distance path and minimum spanning tree belong to P
  - Simply compute an MST and compare its weight to  $B$

# The goal of verification algorithms

- The goal of a verification algorithm is to verify a “yes” answer to a decision problem’s input (i.e., if the answer is “yes” the verification algorithm verifies this answer)
- The inputs to the verification algorithm are:
  - the original input (problem instance) and
  - a *certificate* (possible solution)

# Verification Algorithms

- A *verification algorithm* takes a problem instance  $x$  and answers “yes”, if there **exists** a certificate  $y$  such that the answer for  $x$  with certificate  $y$  is “yes”
- Consider HAMILTONIAN-CYCLE
- A problem *instance*  $x$  lists the vertices and edges of  $G$ :  
 $(\{1,2,3,4\}, \{(3,2), (2,4), (3,4), (4,1), (1,3)\})$
- There **exists** a certificate  $y = (3, 2, 4, 1, 3)$  for which the verification algorithm answers “yes”



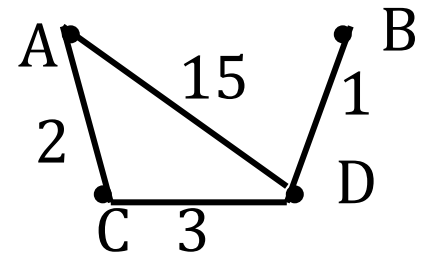
# Polynomial bound verification algorithms

- Given a decision problem  $d$
- A verification algorithm for  $d$  is *polynomial bound* if given an input  $x$  to  $d$ , there exists a certificate  $y$ , such that  $|y| = O(|x|^c)$  where  $c$  is a constant, and a polynomial bound algorithm  $A(x, y)$  that verifies an answer “yes” for  $d$  with input  $x$

Note:  $|y|$  is the size of the certificate,  $|x|$  is the size of the input

# The problem PATH

- PATH denotes the decision problem version of shortest path.
- PATH: Given a graph  $G$ , a start vertex  $u$ , and an end vertex  $v$ . Does there exist a path in  $G$ , from  $u$  to  $v$  of length at most  $k$ ?
- The instance is:  $G=(\{A, B, C, D\}, \{(A, C, 2), (A, D, 15), (C, D, 3), (D, B, 1)\})$   $k=6$
- A certificate  $y=(A, C, D, B)$



# A verification algorithm for PATH

- Verification algorithm:
  - Given the problem instance  $x$  and a certificate  $y$ 
    - Check that  $y$  is indeed a path from  $u$  to  $v$ .
    - Verify that the length of  $y$  is at most  $k$
- Is the verification algorithm for PATH polynomial bound?
- Is the size of  $y$  polynomial in the size of  $x$ ?
- Is the verification algorithm polynomial bound?

# Example: A verification algorithm for TS (Traveling Salesman)

- Given a problem instance  $x$  for TS and a certificate  $y$ 
  - Check that  $y$  is indeed a cycle that includes every vertex exactly once except for the starting node
  - Verify that the length of the cycle is at most  $B$
- Is the size of  $y$  polynomial in the size of  $x$ ?
- Is the verification algorithm polynomial?

# The class NP

## (Non-deterministic Polynomial)

- NP is the class of decision problems for which there is a polynomial bound **verification** algorithm
- It can be shown that:
  - All decision problems in P, and
  - Decision problems such as traveling salesman, knapsack, bin packing, are also in NP

# The relation between P and NP

- $P \mid NP$
- It is not known whether  $P = NP$  or  $P \neq NP$
- Problems in P can be *solved* “quickly”
- Problems in NP can be *verified* “quickly”
- It is easier to verify a solution than solving a problem
- Some researchers believe that P and NP are not the same class (But no one has proved whether or not this is true)

# Polynomial reductions

- **Motivation:** The definition of NP-completeness uses the notion of *polynomial reductions* of one problem A to another problem B, written as

$$A \propto B$$

- Let *tran* be a function that converts any input x for decision problem A into input *tran*(x) for decision problem B

# Polynomial reductions

## **Theorem:**

If  $A \propto B$  and  $B$  is in  $P$ , then  $A$  is in  $P$

If  $A$  is not in  $P$  then  $B$  is also not in  $P$

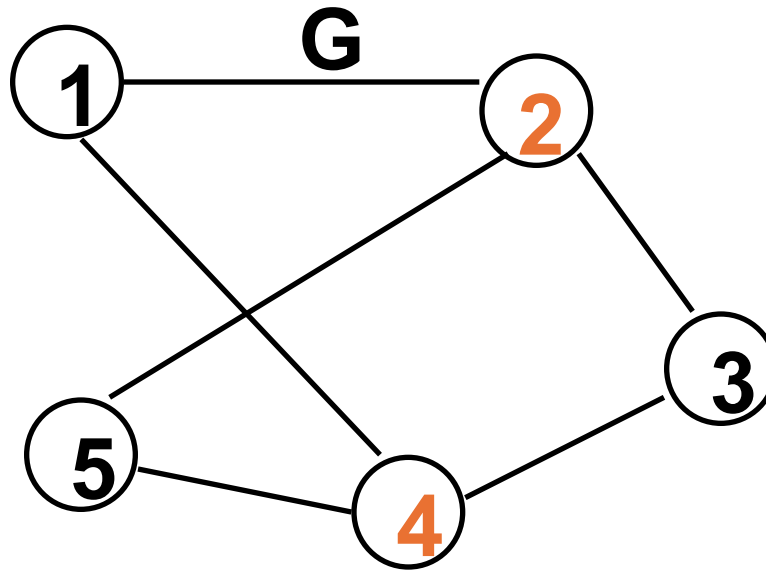
# Vertex Cover

- Reduce clique to vertex cover

# The vertex-cover problem

- A *vertex cover* of an undirected graph is a set of vertices  $V'$  such that for every edge  $(u,v)$ , either  $u$  or  $v$  or both are in  $V'$ . The problem is to find a cover of minimum size.
- VERTEX-COVER
  - Input: **A graph  $G$  and a number  $k$ .**
  - Output: **YES iff  $G$  has a vertex cover of size  $k$ .**

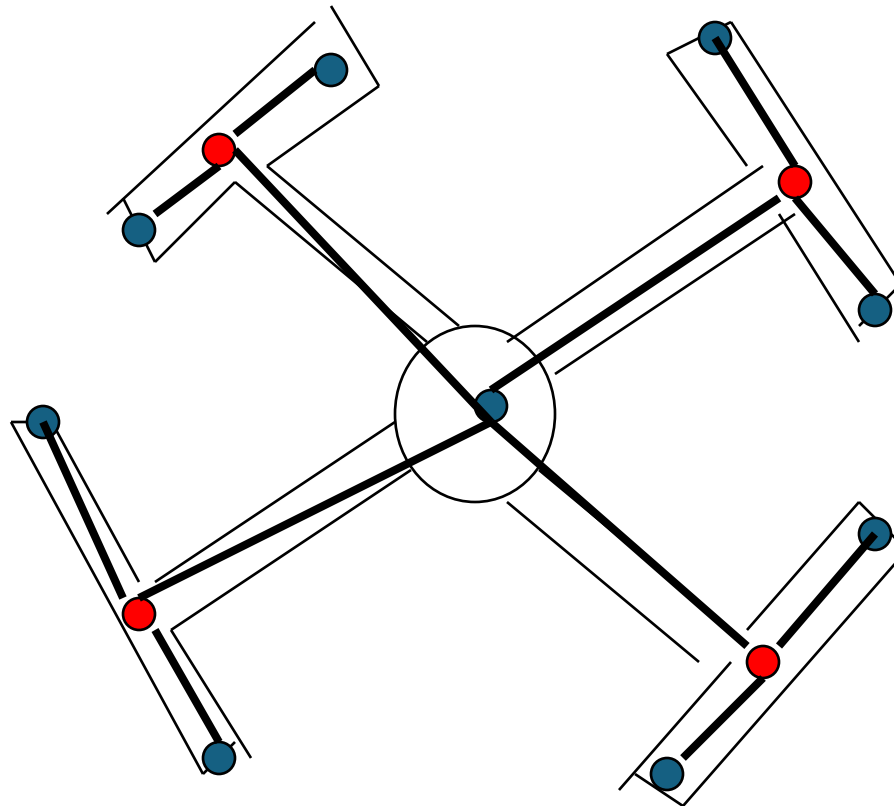
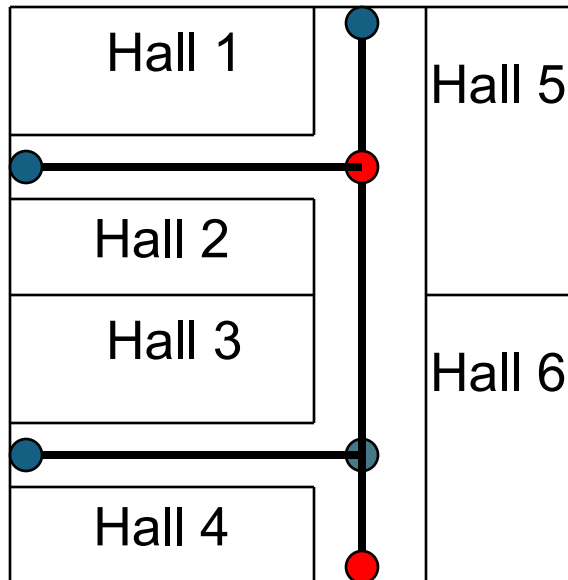
# Example of a vertex cover problem



$k=2$

# Application of vertex cover

- What is the fewest # of guards we need to place in a museum to cover all the corridors? An airport to cover all the main walkways



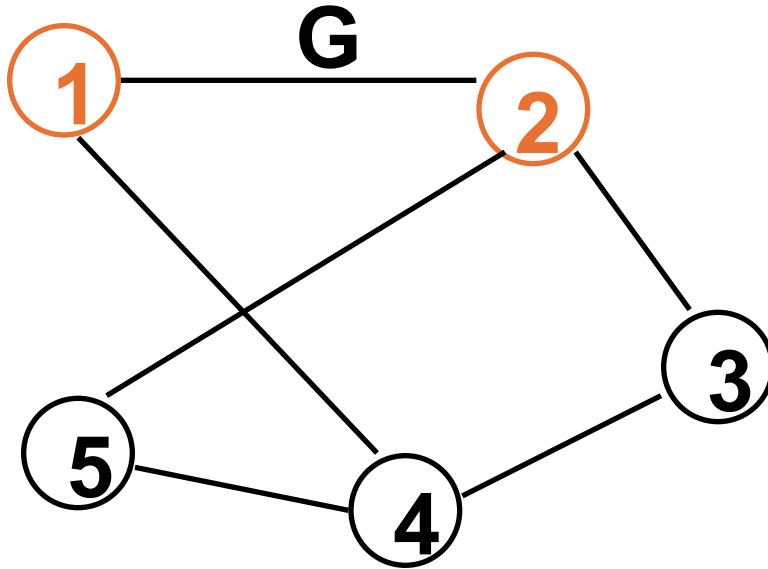
# The vertex-cover problem

- **Theorem:** VERTEX-COVER is NP-complete.
- **Proof: Step 1.** VERTEX-COVER  $\in$  NP (obvious algorithm, given a subset of vertices).
- **Step 2.** We select CLIQUE (will show that CLIQUE  $\propto$  VERTEX-COVER)

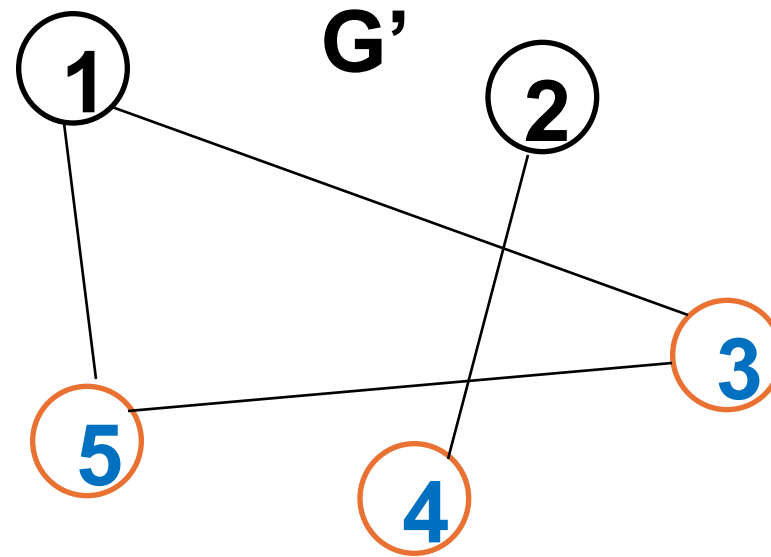
# The reduction

- **Step 3.** The mapping.
- Given an instance of the CLIQUE problem  $\langle G, k \rangle$  we output an instance  $\langle G', |V|-k \rangle$  of the VERTEX-COVER problem.
- $G'$  has the same vertices as  $G$  and exactly those edges that are not in  $G$ .
- It is easy to show the reduction is polynomial (step 5)

# Reduction Example



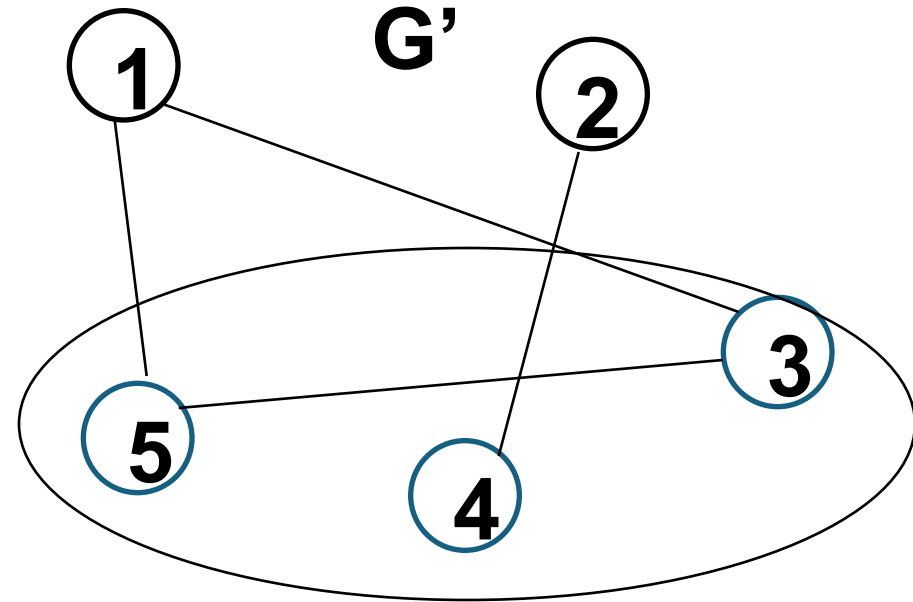
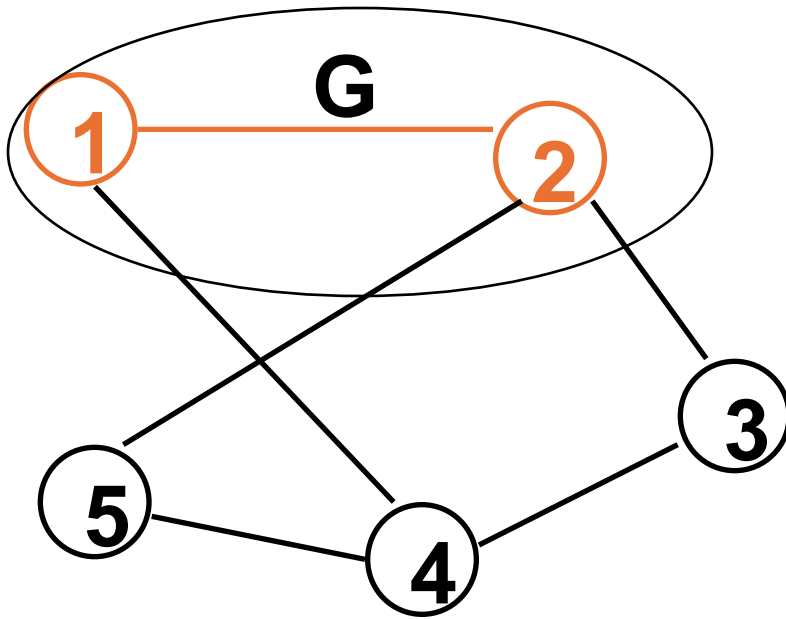
**Clique {1,2} of  
size 2**



**Cover {3,4,5} of  
size 3**

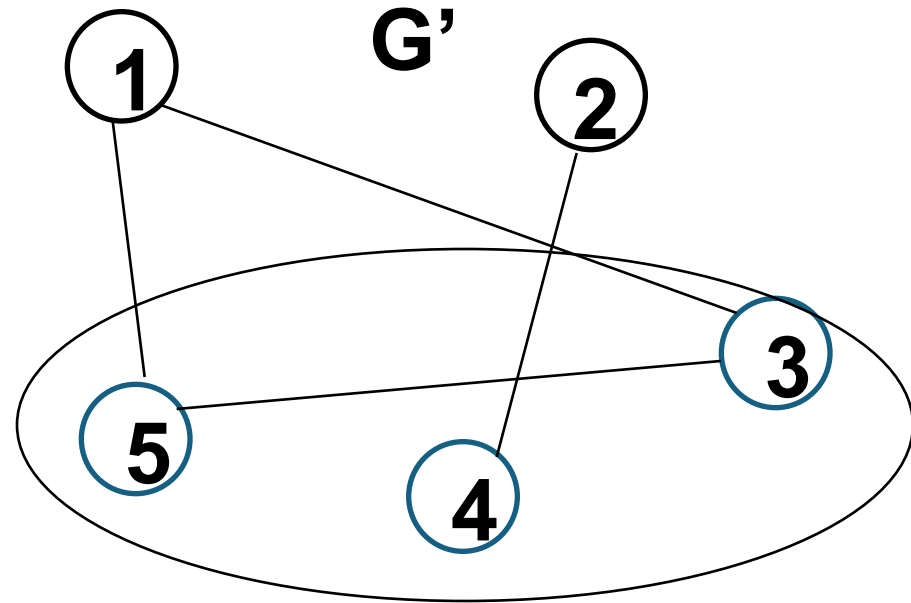
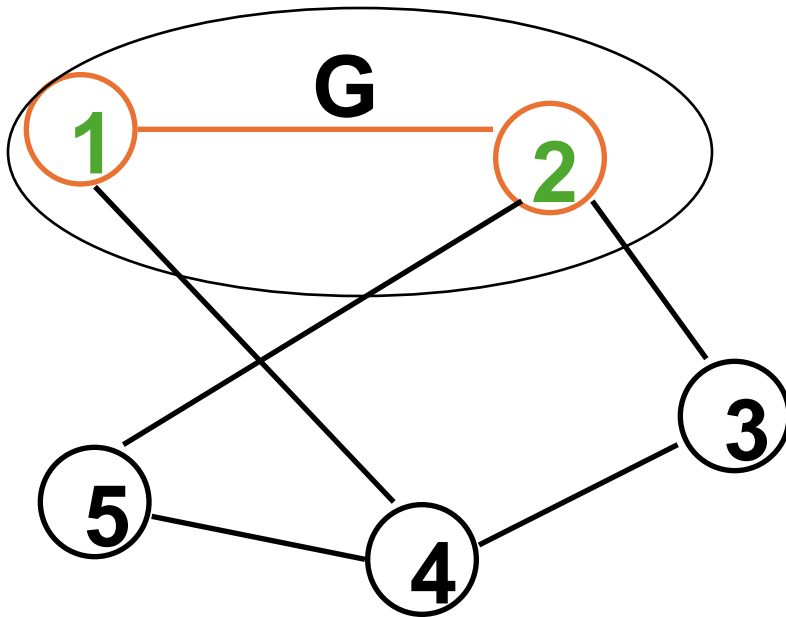
## Step 4. Correctness of the reduction

- Assume  $G$  has a **clique**  $C$  of size  $k$ .
- In  $G'$  there are no edges between any pair of vertices in  $C$



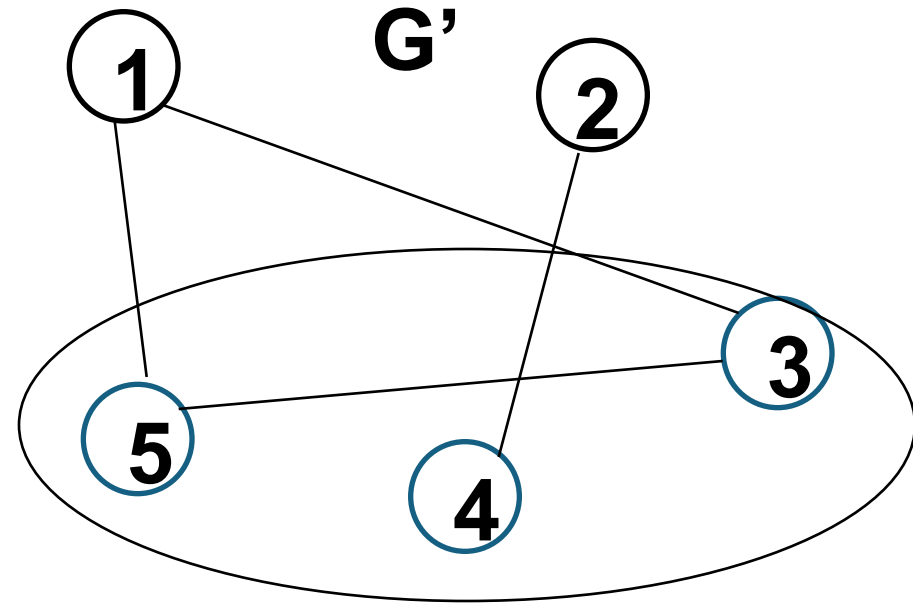
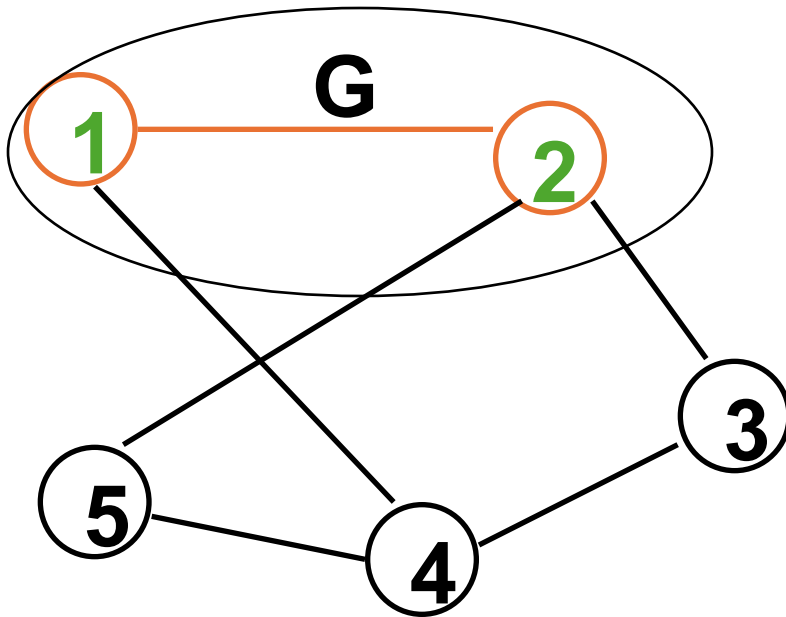
## Step 4 cont

- So all edges in  $G'$  are between a node in  $C$  and a node in  $V-C$ , or two nodes in  $V-C$ .
- So  $V-C$  is a vertex cover for  $G'$ .



# Step 4. Correctness of the reduction

- Assume  $G'=(V, E')$  has a **vertex cover**  $V' \subseteq V$ , where  $|V'| = |V|-k$ .
- Thus for all  $u, v \in V-V'$  (not in the cover),  $(u,v) \notin E'$  and thus  $(u,v) \in E$
- $V-V'$  is thus a clique.



# Homework : Traveling Salesperson

- Reduce Hamiltonian Cycle to Traveling Salesperson

# Traveling Salesman

- A *tour* is a Hamiltonian cycle in a graph. We want the minimum cost tour in a weighted graph.
- **TSP:**
  - **Input:** A graph  $G$ , weights  $c$  for edges and a positive integer  $k$ .
  - **Output:** YES iff  $G$  with weights  $c$  has a TS tour of cost at most  $k$ .

# Traveling Salesman

- **Theorem:** TSP is NP-complete.
- **Proof: Step 1: TSP is in NP**
  - The certificate is a representation of the tour, for example a permutation of the cities.
  - This certificate can be verified easily by checking that all cities are included exactly once and that the sum of the distances between all pairs of consecutive tour nodes is  $k$  or less.
  - This can be done in polynomial time, so  $\text{TSP} \in \text{NP}$ .

# The reduction

- Step 2: Select HAM-CYCLE (We will show that  $\text{HAM-CYCLE} \propto \text{TSP}$ ).
- Step 3: The reduction
  - Given an instance  $G$  of HAM-CYCLE, we construct a graph  $G' = (V, E')$ .  $G'$  is a complete graph and  $c(i,j) = 1$  if  $(i,j)$  is an edge and 2 or infinity otherwise.
  - Find out if there is TSP with length  $n$  where  $n$  is the number of the vertices.

### 3. Reduction Steps

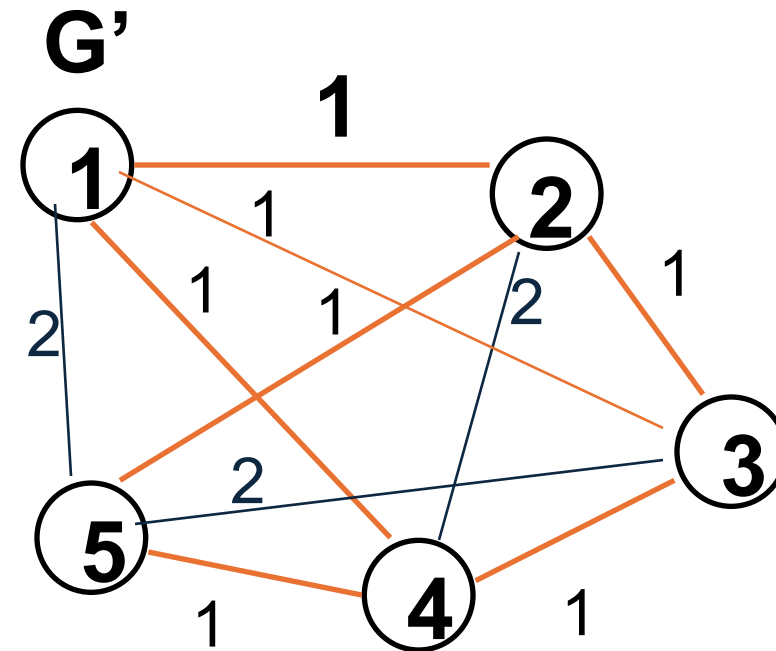
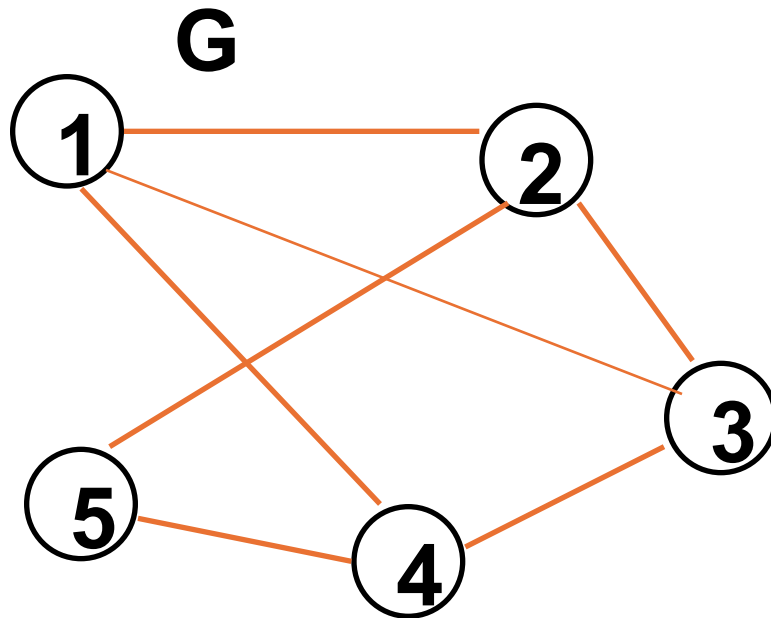
#### Step 1: Construct the Complete Graph $G'$

1. Start with the graph  $G = (V, E)$  from the HC instance.
2. Create a complete graph  $G' = (V, E')$  where every pair of vertices in  $G'$  is connected by an edge.

#### Step 2: Assign Weights to Edges in $G'$

1. For edges  $(u, v)$  that exist in  $G$  (i.e.,  $(u, v) \in E$ ):
  - Assign a weight of  $w(u, v) = 1$ .
2. For edges  $(u, v)$  that do not exist in  $G$  (i.e.,  $(u, v) \notin E$ ):
  - Assign a weight of  $w(u, v) = \infty$  (or a very large value).

# The reduction (example)

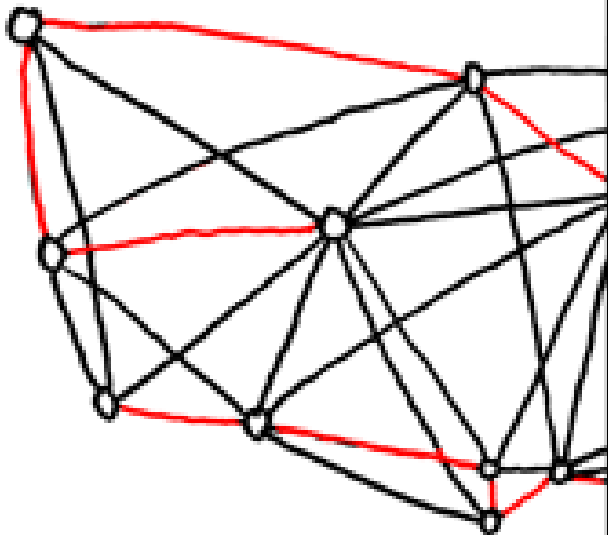


# The reduction (step 4)

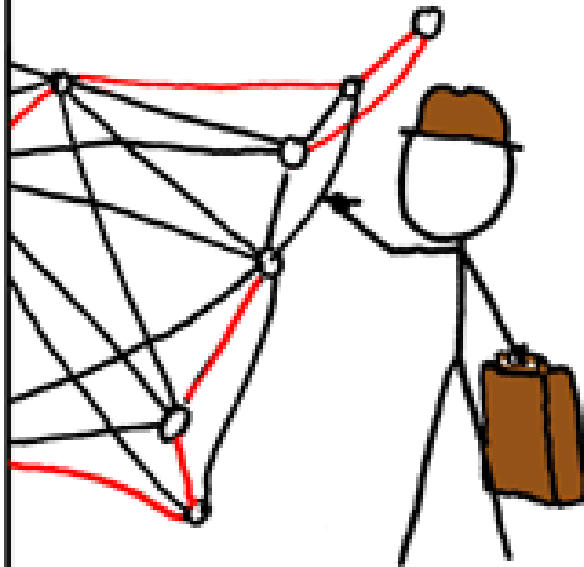
- If  $G$  has a Hamiltonian cycle  $h$ , each edge in  $h$  belongs to  $E$  and thus has no cost in  $G'$ . Thus  $h$  is a tour with cost  $n$ .
- If  $G'$  has a tour of cost  $n$ , the tour must have edges from  $E$  (since any edge not in  $E$  adds 2 to the cost). Thus, the tour must be a Hamiltonian cycle in  $G$ .

# Questions?

BRUTE-FORCE  
SOLUTION:  
 $O(n!)$



DYNAMIC  
PROGRAMMING  
ALGORITHMS:  
 $O(n^2 2^n)$



SELLING ON EBAY:  
 $O(1)$

STILL WORKING  
ON YOUR ROUTE?

SHUT THE  
HELL UP.

